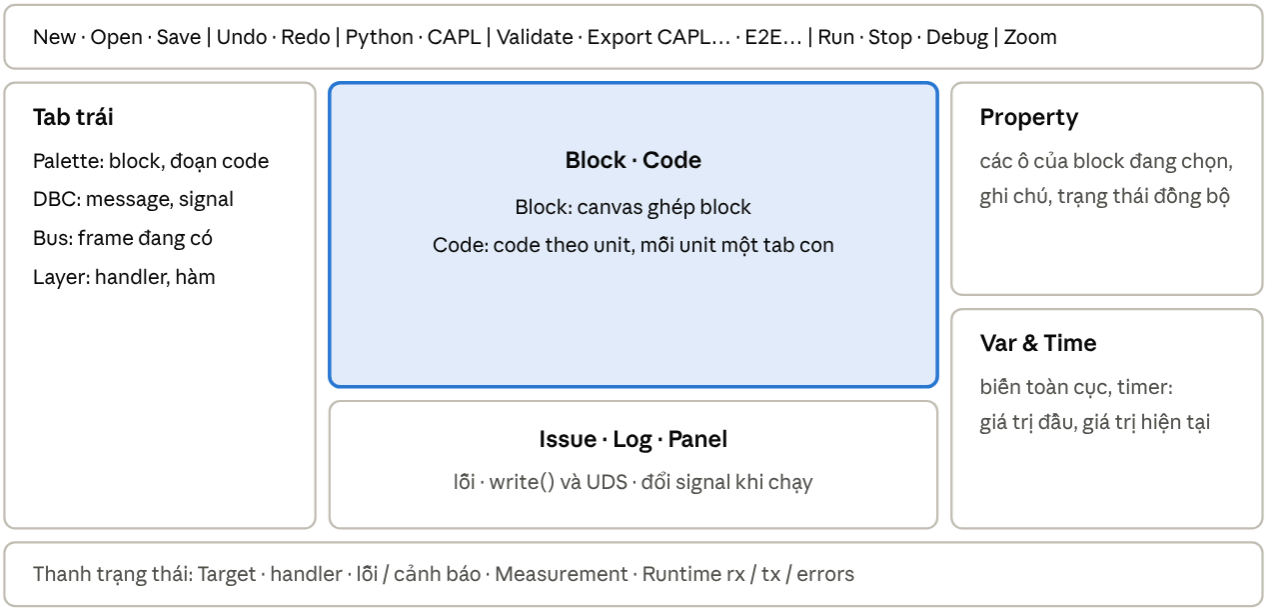
Hướng dẫn sử dụng Program widget

Oct 17, 2026 · @SangNguyen

Cửa sổ Program

Program là nơi viết một chương trình Python (hoặc ghép block) chạy trên Measure Bus: mô phỏng ECU, làm gateway giữa hai network, chạy test UDS. Cùng chương trình đó xuất được sang CAPL để chạy trong CANoe. Chi tiết từng block có trong tài liệu Hướng dẫn các block của Program.

Cửa sổ Program: code ở giữa, công cụ xung quanh



bố cục cửa sổ Program · 7 vùng

Tab trái và panel phải chạy suốt chiều cao; tab Issue / Log / Panel nằm ngay dưới vùng code.

Vùng	Thành phần	Dùng để
Thanh công cụ	New, Open, Save	Tạo, mở, lưu chương trình. Phím tắt lưu là Ctrl+Alt+S; Shift + click Save là Save As
Thanh công cụ	Undo, Redo	Hoàn tác trong tab đang mở (Block hoặc Code)
Thanh công cụ	Python / CAPL	Target. CAPL làm mờ những gì không xuất được và báo trong tab Issue

Vùng	Thành phần	Dùng để
Thanh công cụ	Validate	Kiểm tra code và block, kết quả ở tab Issue
Thanh công cụ	Export CAPL…, E2E…	Xuất file .can; khai báo CRC / Alive Counter của message
Thanh công cụ	Run, Stop, Debug	Chạy / dừng chương trình. Debug bật trước Run để sáng dòng và block đang chạy
Thanh công cụ	−, 100%, +, Fit	Phóng to / thu nhỏ tab Block hoặc Code
Tab trái	Palette	Block (tab Block) hoặc đoạn code mẫu (tab Code). Ô tìm kiếm phía trên
Tab trái	DBC	Message và signal của CAN Database theo từng network. Double-click chèn `send(...)` / `get_signal(...)`; chuột phải message » E2E
Tab trái	Bus	Frame đang có trên từng network sau HOME › Start: số frame, chu kỳ, hướng, giá trị signal. Double-click chèn tên vào code
Tab trái	Layer	Danh sách handler và hàm của chương trình; click để nhảy tới dòng
Giữa	Block	Canvas ghép block
Giữa	Code	Code chia theo unit (tab con). Nút + thêm unit; chuột phải tab unit: Rename, Turn off / on, Move, Delete
Tab dưới	Issue	Lỗi và cảnh báo khi gõ, lỗi lúc chạy. Double-click tới dòng; chuột phải để sửa nhanh
Tab dưới	Log	`write()`, thông báo của runtime, request / phản hồi UDS
Tab dưới	Panel	Biến và signal của các message chương trình đang gửi. Double-click để đổi giá trị hoặc generator khi đang chạy
Phải	Property	Các ô và ghi chú của block đang chọn, trạng thái đồng bộ với tab Code

Vùng	Thành phần	Dùng để
Phải	Var & Time	Biến toàn cục và timer: tên, kiểu, giá trị đầu, giá trị hiện tại khi chạy. Nút + / − thêm, xoá biến
Thanh trạng thái	Target, handler, lõi, Measurement, Runtime	Runtime đếm rx / tx / errors khi chương trình chạy

Một chương trình lưu thành hai file đặt cạnh nhau: `ten.py` (code, mọi unit) và `ten.layout.json` (canvas của tab Block, trạng thái đồng bộ, tuỳ chọn Export CAPL). File `.py` đọc và sửa được bằng bất kỳ editor nào.

Bắt đầu nhanh

Từ lúc nạp DBC tới lúc chương trình đầu tiên chạy trên bus cần 8 bước:

1. **Nạp DBC.** Trong cửa sổ CAN Database, gán DBC cho network (ví dụ DBC C2 cho `CAN1`). Tab DBC của Program tự cập nhật.
2. **Mở Program** từ cửa sổ chính. Tab Code có sẵn một chương trình mẫu.
3. **Viết chương trình** ở tab Code, hoặc ghép block ở tab Block. Gõ tên message và signal đúng như DBC; tab Issue báo ngay khi gõ sai.
4. **Validate.** Tab Issue phải không còn dòng Error. Warning vẫn chạy được.
5. **HOME › Start** để bắt đầu đo trên Measure Bus. Tab Bus bắt đầu hiện frame.
6. **Run.** Thanh trạng thái hiện `Runtime: running`; tab Code chuyển sang chỉ đọc cho tới khi dừng.
7. **Quan sát và điều khiển.** Log hiện `write()`, Var & Time hiện giá trị biến, Panel cho đổi signal đang gửi. Phím bấm khi cửa sổ có focus đi vào `@on_key`.
8. **Stop**, rồi **Save** (Ctrl+Alt+S). HOME › Stop hoặc đóng cửa sổ cũng dừng chương trình.

Chương trình thử với DBC C2: gửi `C2_ESC_Info` theo chu kỳ của DBC và ghi Log mỗi khi IVC đổi % bàn đạp phanh.

```
"""Thử nhanh: ESC gửi C2_ESC_Info, ghi Log khi IVC đổi bàn đạp phanh."""

@on_start
def start():
    set_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC normal")
    send_cyclic("C2_ESC_Info")                # chu kỳ của DBC: 20 ms


@on_signal("C2_IVC_CVC_StsEpt", "C2_IVC_CVC_BrkPedalPct")
def brake(value):
    write("Phanh: %.1f %%", value)
```

Run bị từ chối khi: chưa HOME › Start, code còn lỗi cú pháp hoặc lỗi phân tích, hoặc dòng ngoài cùng của chương trình gây lỗi khi nạp. Lý do nằm ở Log và tab Issue.

Ngôn ngữ của tab Code

Tab Code là Python thường cộng với các hàm của runtime. Dòng ở cấp ngoài cùng chạy một lần khi nạp chương trình (khai báo biến, hằng số, E2E); mọi việc còn lại nằm trong hàm có decorator sự kiện.

Gọi tên message

Cách viết	Nghĩa
`"C2_ESC_Info"`	Message DBC. Chỉ đúng khi một network duy nhất có nó
`"CAN2::C2_ESC_Info"`	Message đó trên network CAN2. Bắt buộc khi hai network nạp cùng DBC
`0x271` hoặc `"CAN2::0x271"`	CAN ID thô, không qua DBC
`"CAN2::*"` hoặc `"*"`, `channel="CAN2"`	Mọi frame của CAN2 (chỉ dùng cho `@on_message`)
`channel="CAN2"` / `network="CAN2"`	Hai tên của cùng một tham số, dùng cho CAN ID thô

Sự kiện

Decorator	Chạy khi	Tham số của hàm
`@on_start`	Sau khi Run, một lần	không
`@on_stop`	Khi Stop / HOME › Stop, trước khi mọi thứ dừng	không
`@on_message(m, channel=, tx=False)`	Mỗi frame nhận được của `m`; `tx=True` nhận cả frame mình gửi	`msg`
`@on_message("*", channel="CAN1")`	Mọi frame CAN1 mà không handler nào khác nhận	`msg`
`@on_signal(m, sig, update=False)`	Signal đổi giá trị; `update=True` là mọi frame	`value`
`@on_timer("t1")`	Timer `t1` hết giờ	không
`@on_key("a")`	Bấm phím khi đang Run; `"F1"`, `"*"` cũng được	không
`@sequence("Tên")`	Sau `@on_start`, lần lượt theo thứ tự trong file, trong luồng riêng	không
`@on_error_frame`	Chưa được Run gọi: core chưa chuyển error frame lên	`frame`

Hàm của runtime

Nhóm	Hàm	Việc
CAN	`send(m, data=None)`	Một frame. `data` là dict signal, list byte, hoặc bỏ trống để gửi ảnh TX
CAN	`send_cyclic(m, period_ms=None, data=None)` / `stop_cyclic(m)`	Gửi theo chu kỳ; bỏ `period_ms` thì lấy chu kỳ của DBC
CAN	`set_signal(m, sig, value)`	Sửa ảnh TX. `value` là số vật lý hoặc chữ của bảng giá trị
CAN	`get_signal(m, sig, rx_only=False, max_age=None, default=None)`	Giá trị mới nhất trên bus

Nhóm	Hàm	Việc
CAN	`last_frame(m)`, `msg_age(m, rx_only=False)`	Frame cuối cùng; tuổi của nó tính bằng ms
CAN	`forward(msg, ch, signals=None, data=None)`	Chuyển frame vừa nhận sang network khác (gateway)
CAN	`set_generator(m, sig, gen, step=None)`	`None`, `"CheckSum"`, `"Alive Counter"`, `"Ramp"`, `"Random"`
E2E	`e2e_config(crc=, alive=)`	CRC và dải Alive mặc định của chương trình
E2E	`e2e_message(m, crc=, alive=, algo=, over=, data_id=, alive_range=)`	Vị trí và cách tính CRC / Alive của một message
E2E	`e2e_enable(m, crc=, alive=)`	Bật / tắt CRC hoặc Alive để tiêm lỗi
Timer, Log	`set_timer(name, ms)`, `cancel_timer(name)`, `write(fmt, *args)`	Timer chạy một lần; `write` dùng định dạng `%`
Chỉ trong `@sequence`	`wait(ms)`, `wait_for_message(m, timeout=)`, `expect_signal(m, sig, value, timeout=)`	Chờ thời gian, chờ frame, chờ signal (ghi OK / FAILED)
Chỉ trong `@sequence`	`uds_request(ecu, req, timeout=None)`, `security_access(ecu, level)`	Request UDS; mở khoá bằng DLL của Diagnostic Setting
UDS	`tester_present(ecu, on)`	Giữ phiên chẩn đoán: `3E 80` mỗi 2 s khi không có request

Frame nhận được (`msg`)

Thuộc tính	Giá trị
`msg.id`, `msg.channel`, `msg.name`, `msg.ref`	CAN ID, network, tên DBC (hoặc `0x...`), `"CAN1::C2_ESC_Info"`
`msg.data`, `msg.dlc`	Các byte dữ liệu và số byte
`msg.byte(n)`, `msg.word(n, big_endian=False)`	Một byte; hai byte ghép (little endian như CAPL)

Thuộc tính	Giá trị
`msg.signal("Sig")`	Giá trị vật lý của signal trong frame
`msg.is_fd`, `msg.brs`, `msg.is_extended`, `msg.is_remote`	Kiểu frame
`msg.dir`, `msg.age_ms`	`"Rx"` / `"Tx"`; số ms từ lúc nhận

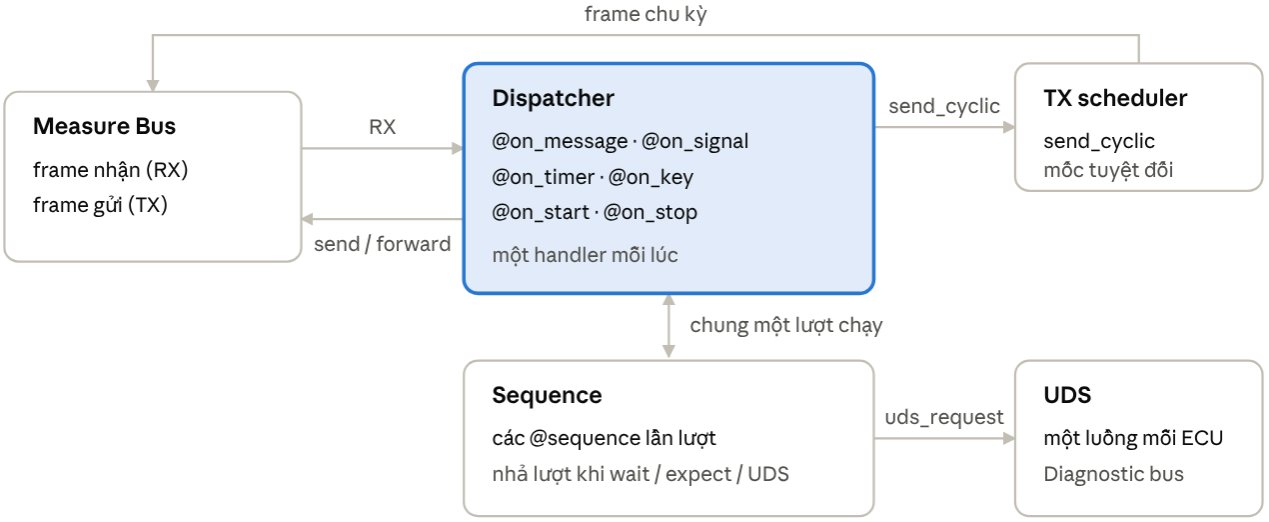
Biến, kiểu và unit

- Gán biến toàn cục trong hàm không cần `global`: runtime tự thêm. Vì vậy tham số của hàm không được trùng tên biến toàn cục.
- Khai báo có kiểu tràn số như C: `byte` (8 bit), `word` (16), `dword` (32), `long` (32 có dấu), `int64` (64 có dấu). Ví dụ `cnt: byte = 250` rồi `cnt += 10` cho 4.
- Unit là các tab con của tab Code, vẫn là một file `.py`. Mỗi unit bắt đầu bằng dòng `# ==== unit: Tên ====`; unit tắt có `(off)` sau tên. Mọi unit dùng chung biến và hàm. Unit tắt vẫn được kiểm tra nhưng không chạy, số dòng giữ nguyên.

Chạy và gỡ lỗi

Handler chạy lần lượt từng cái trên một luồng, giống CAPL. Sequence chạy trên luồng riêng và chỉ nhường chỗ khi đang chờ, nên code của sequence và của handler không bao giờ chạy xen vào nhau.

Handler chạy lần lượt; sequence chỉ chen vào khi đang chờ



các luồng khi Run · dispatcher, sequence, TX scheduler, UDS

Dispatcher và sequence dùng chung một lượt chạy; frame chu kỳ và UDS có luồng riêng nên không bị handler làm trễ.

Các luồng khi Run

Luồng	Chạy gì	Điều cần nhớ
Dispatcher	Mọi handler: `@on_start`, `@on_message`, `@on_signal`, `@on_timer`, `@on_key`, `@on_stop`	Một handler mỗi lúc. Handler chạy quá 20 ms làm trễ handler khác; Log cảnh báo một lần
Sequence	Các `@sequence`, lần lượt theo thứ tự trong file, sau `@on_start`	Được chờ: `wait`, `wait_for_message`, `expect_signal`, `uds_request`, `security_access`
TX scheduler	Các frame `send_cyclic`	Chu kỳ tính theo mốc tuyệt đối, không trôi theo thời gian chạy handler
UDS	Mỗi ECU dùng trong chương trình một luồng, trên Diagnostic bus	Tester Present cũng chạy ở đây

Công cụ khi chạy

Chỗ	Cho biết	Cách dùng
Tab Issue	Lỗi phân tích (0,4 s sau khi gõ) và lỗi lúc chạy, ví dụ `Run · on_req() · line 26 · IndexError (×2)`	Double-click tới đúng dòng và unit. Chuột phải khi dòng có `[right-click: fix]`
Tab Log	`write()` kèm thời gian từ lúc Run, thông báo runtime, mỗi request UDS và phản hồi	`expect_signal` ghi `OK after 35 ms` hoặc `FAILED after 1000 ms (value 0)`
Var & Time › Current	Giá trị biến toàn cục và trạng thái timer, cập nhật mỗi 200 ms	Double-click tới dòng khai báo
Tab Panel	Biến và signal của message chương trình đang gửi, kèm generator	Double-click giá trị để đổi khi đang chạy, thay cho panel / system variable của CANoe
Debug	Dòng (tab Code) và block (tab Block) vừa chạy sáng lên	Bật trước khi Run. Handler chạy chậm hơn một chút khi bật
Thanh trạng thái	`Runtime: running · rx … · tx … · errors …`	`errors` tăng là có lỗi mới ở tab Issue

Khi có lỗi lúc chạy

- Handler gặp lỗi dừng ở dòng đó; các handler khác vẫn chạy. Lỗi giống nhau được gộp và đếm.
- Sequence gặp lỗi thì dừng sequence đó, sequence kế tiếp vẫn chạy.
- Stop: `@on_stop` chạy trước; sequence đang chờ (`wait`, request UDS) bị ngắt ngay; kết nối UDS đóng lại.
- Lỗi thường gặp và cách sửa có ở mục cuối tài liệu.

E2E: CRC và Alive Counter

Mỗi frame của message có E2E được tính trước khi gửi: Alive Counter ghi trước, sau đó CRC tính trên kết quả. Có hai nguồn khai báo:

- **Comment trong DBC.** Signal có comment kiểu `Checksum` / `Alive counter` (hoặc tên `CRC_...` / `ALIVE_...`) tự nhận generator, tính theo E2E Config của Send CAN hoặc `e2e_config()` của chương trình.
- **`e2e_message()` trong chương trình.** Chỉ rõ vị trí, thuật toán, các byte tính CRC, Data ID, dải Alive. Khai báo này thắng generator của DBC cho message đó, áp dụng cả cho frame gửi theo CAN ID, và được Export CAPL mang theo.

Khai báo

```
e2e_config(crc="CRC-8/SAE-J1850", alive="0-14")      # mặc định của chương
trình
e2e_message("C2_ESC_ABS_Status", crc="CRC_C2_ESC_ABS_Status",
alive="ALIVE_C2_ESC_ABS_Status")
e2e_message(0x7BA, network="CAN1", crc="bytes6-7 big", algo="CRC-16/CCITT-
FALSE", data_id=0x1234)
```

Viết ở cấp ngoài cùng của file; thứ tự giữa `e2e_config` và `e2e_message` không quan trọng.

Tham số	Giá trị	Ví dụ
`crc`, `alive`	Tên signal DBC, hoặc vị trí trong frame. Bỏ một trong hai nếu message chỉ có một	`"CRC_C2_ESC_Info"`, `"byte0"`, `"bytes6-7"` (byte 6 là byte thấp), `"bytes6-7 big"` (byte 6 là byte cao), `"byte1 bit0-3"`, `"byte1 bit4"`

Tham số	Giá trị	Ví dụ
`algo`	Tên CRC có sẵn, Custom CRC của E2E Config, hoặc tham số. Bỏ trống: CRC của chương trình	`"CRC-8/SAE-J1850"`, `"CRC-8/AUTOSAR"`, `"CRC-16/CCITT-FALSE"`, `"CRC-16/XMODEM"`, `"CRC-16/GENIBUS"`, `"XOR-8"`, `"ADD-8"`, `{"width": 8, "poly": 0x2F, "init": 0xFF, "xorout": 0xFF}`
`over`	Các byte tính CRC. Byte của chính CRC luôn bị bỏ ra	`"all"` (mặc định), `"1-7"`, `"0-3,5-7"`
`data_id`	Byte đặt trước dữ liệu khi tính CRC, không gửi ra bus	`0x12`, `0x1234` (2 byte, byte thấp trước), `"34 12"`
`alive_range`	Dải đếm. Bỏ trống: dải của chương trình	`(0, 14)`, `(1, 9, 2)` (bước 2), `"0-14"`
`network` / `channel`	Network của CAN ID thô	`"CAN1"`

Hộp thoại E2E…

Hộp thoại viết sẵn dòng `e2e_message(...)`, không cần gõ tay:

1. Mở bằng nút **E2E…**, hoặc chuột phải một message ở tab DBC » **E2E of …**.
2. Chọn message DBC, hoặc **CAN ID…** rồi nhập ID hex, network và số byte.
3. Đánh dấu trên lưới bit (hàng = byte, cột = bit 7..0): chọn ô rồi bấm **CRC ← selection** hoặc **Alive ← selection**. Double-click một signal để chọn trọn signal; vùng chọn trùng một signal thì code ghi tên signal đó.
4. Chọn CRC, Over bytes, Data ID; với Alive chọn **own range** nếu khác dải của chương trình.
5. Xem dòng mô tả, 4 frame xem trước và check value của CRC. Dòng đỏ là lỗi, dòng vàng là cảnh báo (ví dụ CRC 16 bit trong chỗ 8 bit).
6. **Write to code** ghi hoặc thay dòng của message đó (một bước Undo). **Remove from code** xoá dòng.

Tiêm lỗi E2E

Mục đích	Code
CRC sai	`e2e_enable("C2_ESC_ABS_Status", crc=False)`, bật lại bằng `crc=True`

Mục đích	Code
Alive đứng yên	`e2e_enable("C2_ESC_ABS_Status", alive=False)`; bật lại thì đếm lại từ đầu dải
CRC sai trên frame forward	`forward(msg, "CAN2", {"CRC_C2_ESC_VAFS": msg.byte(0) ^ 0xFF})`
Đổi trên tab Panel khi đang chạy	Đặt generator của signal CRC / Alive về `None` khi đang chạy

Forward có sửa signal thì CRC được tính lại theo khai báo của network đích (hoặc của network nguồn), còn Alive giữ nguyên như ECU gửi.

Mô phỏng ECU (CAN và CAN FD)

Mô phỏng ECU là cho chương trình đóng vai một node vắng mặt: gửi đúng message, đúng chu kỳ, có E2E, và phản ứng với các node khác. File mẫu đầy đủ cho ESC trên bus C2 là `c2_esc_simulation.py`.

Quy trình

1. **Liệt kê message.** Ở tab DBC, tìm message node cần mô phỏng gửi (TX) và message nó nhận (RX). Với ESC trên C2: TX là `C2_ESC_ABS_Status`, `C2_ESC_Info`, `C2_ESC_VAFS`; RX là các message của IVC.
2. **Khai báo E2E** cho từng message TX (mục E2E ở trên).
3. **Giá trị ban đầu và chu kỳ** trong `@on_start`: `set_signal(...)` rồi `send_cyclic("Msg")` để lấy chu kỳ của DBC.
4. **Phản ứng với RX**: `@on_signal` khi chỉ cần biết giá trị đổi; `@on_message` khi đọc nhiều signal của cùng một frame.
5. **Hành vi theo thời gian** (kẹp phanh 800 ms, đèn nháy …): đặt `set_timer`, làm phần sau trong `@on_timer`. Không chờ trong handler.
6. **Giám sát timeout của RX**: timer lặp 100 ms kiểm tra `msg_age("Msg", rx_only=True)`.
7. **Tiêm lỗi bằng tay**: `@on_key` cho CRC sai, Alive đứng, lỗi ECU; hoặc đổi trực tiếp trên tab Panel.
8. **Kịch bản tự chạy**: đặt `@sequence` trong unit riêng (ví dụ `Scenario`); tắt unit khi chỉ cần mô phỏng.
9. **Kiểm tra**: HOME › Start, Run, rồi xem tab Bus — chu kỳ đo được phải bằng chu kỳ DBC, giá trị signal đúng.

Khung chương trình

```
"""ESC simulation — bus C2: mục đích, network, phím, giả định."""

# E2E
e2e_config(crc="CRC-8/SAE-J1850", alive="0-14")
e2e_message("C2_ESC_ABS_Status", crc="CRC_C2_ESC_ABS_Status",
alive="ALIVE_C2_ESC_ABS_Status")
e2e_message("C2_ESC_Info", crc="CRC_C2_ESC_Info", alive="ALIVE_C2_ESC_Info")

# hằng số, biến
STANDSTILL_KMH = 0.5
speed = 0.0


def set_speed(kmh):
    """Hàm phụ: không chờ, chỉ đặt signal."""
    speed = max(0.0, min(float(kmh), 300.0))
    set_signal("C2_ESC_ABS_Status", "C2_ESC_VehicleSpeed", speed)
    set_signal("C2_ESC_ABS_Status", "C2_ESC_VehicleStandstill", 1 if speed <
STANDSTILL_KMH else 0)


@on_start
def start():
    set_speed(0)
    set_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC normal")
    send_cyclic("C2_ESC_ABS_Status")             # 20 ms theo DBC
    send_cyclic("C2_ESC_Info")                   # 20 ms
    set_timer("t_monitor", 1000)


@on_message("C2_MHU_CarMode_Switch")
def car_mode(msg):
    sw = msg.signal("C2_MHU_ESCOFF_Sw")
    if sw == 2:
        set_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC-OFF")
    elif sw == 1:
        set_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC normal")


@on_timer("t_monitor")
def monitor():
    age = msg_age("C2_IVC_CVC_StsEpt", rx_only=True)
    set_signal("C2_ESC_Info", "C2_ESC_Fault_Status", 1 if age is None or age
> 200 else 0)
```

```
    set_timer("t_monitor", 100)


@on_key("c")
def key_crc():
    e2e_enable("C2_ESC_ABS_Status", crc=False)


# ==== unit: Scenario ====
@sequence("ESC scenario")
def scenario():
    for kmh in range(0, 61):
        set_speed(kmh)
        wait(100)
```

Khi bus là CAN FD

Chương trình mô phỏng giữ nguyên; khác biệt nằm ở DBC và cấu hình network.

Điểm	CAN classic	CAN FD
Network	Không cần gì thêm	Network phải bật chế độ FD (data bitrate) trong cấu hình kênh
Message gửi theo tên DBC	Classic	FD có BRS nếu DBC khai báo message là FD; message classic vẫn classic trên network FD
Message gửi theo CAN ID	Tối đa 8 byte	Luôn là FD có BRS, tối đa 64 byte, đệm `00` lên 12, 16, 20, 24, 32, 48, 64
E2E	Thường CRC-8 trong byte 0	Frame dài thường dùng CRC-16 kèm Data ID, ví dụ `crc="bytes0-1 big"`, `algo="CRC-16/CCITT-FALSE"`, `data_id=0x1A0` — theo spec E2E của message
Lỗi hay gặp	—	`is a CAN FD message but network … is not in FD mode`: DBC nói FD mà network chưa bật FD

Ví dụ mô phỏng một frame FD 32 byte gửi theo CAN ID, có CRC-16 và Alive (CAN ID, network và thuật toán chỉ là ví dụ):

```
e2e_message(0x1A0, network="CANFD1", crc="bytes0-1 big", alive="byte2 bit0-
3",
            algo="CRC-16/CCITT-FALSE", data_id=0x1A0, alive_range=(0, 14))

@on_start
def start():
    send_cyclic(0x1A0, 10, [0] * 32, channel="CANFD1")
```

Gateway giữa hai network

Gateway là khi tool đứng giữa hai phần của một bus (hoặc hai bus khác nhau) và chuyển frame qua lại, có thể sửa, chặn, hoặc làm sai frame để thử ECU. File mẫu là `c2_gateway_esc_ivc.py`: CAN1 là phía ESC, CAN2 là phía IVC.

Chuẩn bị

1. Cắt bus: ECU A nối kênh 1 của interface, ECU B nối kênh 2. Mỗi kênh là một network trong cấu hình kênh.
2. Trong CAN Database, nạp DBC cho cả hai network (cùng DBC nếu là một bus bị cắt đôi).
3. Trong code, luôn viết tên có network: `"CAN1::C2_ESC_Info"`. Thiếu phần network thì tab Issue báo lỗi và cho sửa bằng chuột phải.

Cách định tuyến

- `@on_message("*", channel="CAN1")` nhận mọi frame của CAN1 **chưa có handler riêng**. Handler cho một message cụ thể (`@on_message("CAN1::C2_ESC_Info")`) thắng handler `"*"`, giống `on message CAN1.*` của CAPL.
- Handler chỉ nhận frame do ECU gửi (Rx). Frame tool vừa forward hiện lại dạng Tx và không vào handler, nên không tạo vòng lặp.
- Mỗi frame đi qua dispatcher của Python, độ trễ cỡ mili-giây. Cần độ trễ thấp và ổn định thì Export CAPL và chạy gateway trong CANoe.

Muốn	Code	CRC / Alive
Chuyển nguyên vẹn	`forward(msg, "CAN2")`	Giữ nguyên của ECU
Sửa signal	`forward(msg, "CAN2", {"C2_ESC_VehicleSpeed": 120})`	CRC tính lại, Alive giữ
Ghi đè byte	`forward(msg, "CAN2", data=[...])`	Không tính lại gì

Muốn	Code	CRC / Alive
Làm sai CRC	`forward(msg, "CAN2", {"CRC_C2_ESC_VAFS": msg.byte(0) ^ 0xFF})`	CRC nằm trong danh sách sửa nên không tính lại
Chặn message	Không gọi `forward`	Phía bên kia thấy timeout
Đổi CAN ID	`send(0x3A0, msg.data, channel="CAN2")`	Theo `e2e_message` của ID mới (nếu có)

Kiểu frame khi qua gateway (CAN / CAN FD)

Frame nhận	Network đích	Frame ra
Classic	Classic	Classic, nguyên vẹn
Classic	FD	Vẫn là classic
FD	FD	FD, BRS như lúc nhận
FD, ⩽ 8 byte	Classic	Thành classic
FD, > 8 byte	Classic	Không gửi được: handler báo lỗi `cannot go on network …, which is not in FD mode`

ID, 11/29 bit và DLC luôn giữ như frame nhận. Frame RTR (remote) chỉ có trên CAN classic và được giữ.

Ví dụ 1: CAN ↔ CAN, ESC và IVC trên bus C2

```
e2e_config(crc="CRC-8/SAE-J1850", alive="0-14")
e2e_message("CAN2::C2_ESC_ABS_Status", crc="CRC_C2_ESC_ABS_Status",
alive="ALIVE_C2_ESC_ABS_Status")

fake_speed = False


@on_message("*", channel="CAN1")
def esc_to_ivc(msg):
    forward(msg, "CAN2")


@on_message("*", channel="CAN2")
def ivc_to_esc(msg):
    forward(msg, "CAN1")
```

```
@on_message("CAN1::C2_ESC_ABS_Status")
def abs_status(msg):
    if fake_speed:                                      # phím 1 bật / tắt
        forward(msg, "CAN2", {"C2_ESC_VehicleSpeed": 120,
"C2_ESC_VehicleStandstill": 0})
    else:
        forward(msg, "CAN2")


@on_key("1")
def key_speed():
    fake_speed = not fake_speed
    if fake_speed:                                      # mỗi write() một câu cố
định: Export CAPL dịch được
        write("Tốc độ giả 120 km/h: ON")
    else:
        write("Tốc độ giả 120 km/h: OFF")
```

Ví dụ 2: CAN FD ↔ CAN

CAN2 là network FD, CAN1 là classic. Frame FD dài hơn 8 byte không sang được CAN1 nên bị bỏ và đếm; chiều CAN1 → CAN2 luôn đi được.

```
n_dropped = 0


@on_message("*", channel="CAN2")
def fd_to_classic(msg):
    if msg.is_fd and msg.dlc > 8:
        n_dropped += 1
        return
    forward(msg, "CAN1")


@on_message("*", channel="CAN1")
def classic_to_fd(msg):
    forward(msg, "CAN2")                                # vẫn là frame classic
trên CAN2


@on_stop
def stop():
    write("Bỏ %d frame FD dài hơn 8 byte", n_dropped)
```

Ví dụ 3: CAN FD ↔ CAN FD, sửa signal trong frame dài

Hai network đều FD. Frame giữ FD và BRS; khi sửa signal, CRC được tính lại theo `e2e_message` của network đích, kể cả CRC-16 có Data ID. Tên message và signal dưới đây là ví dụ:

```
e2e_message("CAN2::ADAS_Status", crc="CRC_ADAS_Status",
alive="ALIVE_ADAS_Status",
            algo="CRC-16/CCITT-FALSE", data_id=0x2F0)


@on_message("CAN1::ADAS_Status")
def adas(msg):
    forward(msg, "CAN2", {"ADAS_TargetSpeed": 0})     # ép 0 để thử ECU phía
CAN2
```

Test UDS

Test UDS chạy trong `@sequence`: gửi request tới ECU, kiểm tra phản hồi, ghi PASS / FAIL vào Log. Thường kèm một phần mô phỏng các node khác để ECU không ghi DTC mất giao tiếp. File mẫu là `c2_esc_uds_test.py`.

Chuẩn bị

1. Trong **Diagnostic Setting**, ECU cần test (ví dụ `ESC`) phải có: ID request / response, kênh Diagnostic, DLL Security Access nếu cần mở khoá.
2. Mở Program từ cửa sổ chính để Program nhận Diagnostic Setting. Ô ECU của block UDS liệt kê các ECU này.
3. HOME › Start. UDS chạy trên kênh Diagnostic, còn mô phỏng chạy trên Measure Bus.

Kết quả của một request

`r = uds_request("ESC", "22 F1 90")` trả về một kết quả; request nhiều frame (ISO-TP) được xử lý sẵn.

Thuộc tính	Giá trị
`r.positive` / `bool(r)`	`True` khi phản hồi dương
`r.nrc`	Mã NRC khi phản hồi âm, ví dụ `0x31`
`r.hex()`	`"62 F1 90 4C 56 ..."` hoặc `"7F 22 31"`
`r.byte(n)`, `r.data`	Byte thứ n (byte 0 là SID phản hồi), toàn bộ byte

Thuộc tính	Giá trị
`r.ascii(3)`	Phần từ byte 3 thành chữ: VIN, số part, phiên bản
`r.error`	Lý do khi không có phản hồi: timeout, ECU không có trong Diagnostic Setting, kênh chưa nối

Log ghi mỗi request, ví dụ `UDS ESC 22 F1 90 -> 62 F1 90 ... (12 ms)` hoặc `UDS ESC 22 F1 90 -> FAILED: ...`.

Cấu trúc một test

- **Unit Main:** mô phỏng các node mà ECU cần thấy (rest-bus), có E2E.
- **Unit UDS test:** biến đếm PASS / FAIL, các hàm phụ **chỉ đánh giá kết quả**, và một `@sequence`.
- Mọi lệnh chờ (`uds_request`, `security_access`, `wait`, `wait_for_message`, `expect_signal`) gọi thẳng trong `@sequence`. Hàm phụ gọi `uds_request` thì tab Issue báo Error và Run từ chối.

```
# ==== unit: UDS test ====
ECU = "ESC"
n_pass = 0
n_fail = 0


def step(name, ok, detail=""):
    if ok:
        n_pass += 1
    else:
        n_fail += 1
    write("%s  %s  %s", "PASS" if ok else "FAIL", name, detail)
    return ok


def check_positive(name, r):
    return step(name, r.positive, r.hex() or r.error)


def check_nrc(name, r, code):
    return step(name, not r.positive and r.nrc == code, r.hex() or r.error)


@sequence("ESC UDS test")
def uds_test():
    wait(1000)                                                  # ECU khởi động,
```

```
thấy rest-bus
    check_positive("10 03 extended session", uds_request(ECU, "10 03"))
    tester_present(ECU, True)
    step("27 security access level 1", bool(security_access(ECU, 1)))
    r = uds_request(ECU, "22 F1 90")
    step("22 F1 90 VIN", r.positive, r.ascii(3) if r.positive else (r.hex()
or r.error))
    check_nrc("22 FF FF -> NRC 31", uds_request(ECU, "22 FF FF"), 0x31)
    check_positive("14 FF FF FF clear DTC", uds_request(ECU, "14 FF FF FF"))
    tester_present(ECU, False)
    write("===== %d PASS, %d FAIL =====", n_pass, n_fail)
```

Các bước test hay dùng

Request	Kiểm tra	Ghi chú
`10 01`, `10 03`	Phản hồi `50 01` / `50 03`	Sau `10 03` bật `tester_present` để phiên không hết hạn
`22 F1 90`, `22 F1 87`, `22 F1 95`	`r.positive`, `r.ascii(3)`	DID theo Diagnostic Spec của ECU
`22 F1 86`	`r.byte(3)` = phiên hiện tại	01 default, 03 extended
`22 FF FF`, `27 01` ở default session	`r.nrc` = `0x31`, `0x7F`	Kiểm tra ECU từ chối đúng mã
`security_access(ECU, 1)`	`bool(...)`	`27 01` seed, key từ DLL, `27 02`
`14 FF FF FF`	`r.positive`	Xoá mọi DTC
`19 02 09`	Danh sách DTC từ byte 3, mỗi DTC 4 byte (3 byte mã + status)	Mask `09` = testFailed + confirmed
`11 01`	`r.positive`, rồi `wait_for_message` một message của ECU	ECU khởi động lại, về default session

Kết hợp với mô phỏng và kiểm tra signal

- **DTC mất giao tiếp:** `stop_cyclic("C2_IVC_CVC_StsEpt")`, `wait(2000)`, đọc `19 02 09` phải thấy DTC mới; rồi `send_cyclic(...)` lại và `14 FF FF FF`.

- **Chức năng:** đổi input bằng `set_signal(...)` rồi `expect_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC-OFF", timeout=1000)`; kết quả `True` / `False` đưa vào `step(...)`.
- **Lỗi E2E:** `e2e_enable(..., crc=False)` trên message IVC giả, chờ, rồi đọc DTC của ECU.

UDS trên CAN FD

Code test không đổi. Frame UDS là FD hay classic do cấu hình kênh Diagnostic của ECU quyết định, không do Program. Phần mô phỏng rest-bus trên bus FD theo mục Mô phỏng ECU ở trên.

Export CAPL

Export CAPL viết chương trình thành node CAPL (`.can`) cho CANoe, dùng cùng DBC. Phạm vi là gửi CAN và mô phỏng: handler nào dùng thứ chỉ có trong Python thì bị bỏ ra và liệt kê kèm lý do, phần còn lại vẫn được xuất.

Hộp thoại

Phần	Dùng để
Dòng tóm tắt	Số handler đã xuất / tổng, tên các file `.can`
Nodes	`One node per unit` (mỗi unit một file) hoặc `One node for the whole program`
Tab node	Code CAPL của từng node, tô màu, chỉ đọc
Networks → CANoe channels	Kênh CANoe của mỗi network: `on message CAN<n>.*`, `msg.can = n`
CRC functions	Mỗi CRC của `e2e_message()` mà node dùng. Để trống: node tự viết hàm CRC. Điền lời gọi, ví dụ `crc8({buf}, {len})`, để dùng hàm của thư viện riêng
Include	File `.cin` chứa hàm CRC riêng, ví dụ `..\1_Include_file\crc_cal.cin`
Not exported	Handler bị bỏ và lý do; double-click tới dòng trong tab Code
Notes	Những chỗ chạy khác trong CANoe

Phần	Dùng để
Refresh, Copy, Save…	Xuất lại; chép node đang xem; lưu (một node: một file, nhiều node: vào một thư mục)

File lưu dạng UTF-8, CRLF, dòng đầu `/*@!Encoding:65001*/`. Tuỳ chọn kênh, nodes, CRC functions được lưu cùng chương trình trong `ten.layout.json`.

Python được viết thành gì

Python	CAPL
`@on_start`, `@on_stop`	`on start`, `on preStop`
`@on_message("CAN1::Msg")`, `"*"` + channel	`on message CAN1.Msg`, `on message CAN1.*` (bỏ frame của chính node)
`@on_signal("Msg", "Sig")`	`on signal_change Msg::Sig` (`update=True`: `on signal_update`)
`@on_timer("t")`, `set_timer`, `cancel_timer`	`msTimer t`, `on timer t`, `setTimer`, `cancelTimer`
`@on_key("a")`	`on key 'a'`
`send("Msg")`, `send_cyclic("Msg", ms)`	`send_Msg()` (Alive, CRC, `output`), timer `setTimerCyclic`
`send(0x123, [...])`	`output` của frame thô; trên network FD (cột **FD** của Export CAPL): luôn là CAN FD + BRS, đệm lên 8/12/…/64 byte như Run; trên network classic: > 8 byte thành CAN FD (`FDF`, `BRS`, mã DLC)
`set_signal`, `get_signal`, `msg.signal`	`tx_Msg.Sig = v`, `$Msg::Sig`, `this.Sig`
`forward(msg, "CAN2", {...})`	Bản sao của `this`, `.can = 2`, sửa signal, CRC tính lại, `output`
`e2e_message`, `e2e_enable`, `set_generator`	Hàm CRC, code Alive / CRC trong hàm gửi, cờ bật / tắt
`cnt: byte = 0`, `x = 0`	`byte cnt = 0;`, `long x = 0;`

Không xuất được (chỉ chạy bằng Run)

`@sequence`, `wait`, `wait_for_message`, `expect_signal`, mọi hàm UDS, `last_frame`, `msg_age`, generator `Ramp` / `Random`, dict, chuỗi lưu trong biến, `try` / `with` / `lambda`. Chọn target **CAPL** trên thanh công cụ để tab Issue báo trước những chỗ này.

Với CAN FD, node CAPL gửi frame giống Run khi network được tick ở cột **FD** của Export CAPL (tool tự tick theo cấu hình kênh, hoặc khi DBC của network có message CAN FD). Khi đó frame gửi theo CAN ID là CAN FD + BRS, đệm lên 8/12/…/64 byte, và `forward` sang network classic tự chuyển frame FD ≤ 8 byte thành classic. Gửi theo tên DBC thì CANoe lấy dạng frame từ DBC. Giám sát timeout bằng `msg_age` không xuất được; trong CANoe dùng cơ chế timeout của CAPL.

Lỗi thường gặp và cách sửa

Thông báo hiện ở tab Issue (khi gõ hoặc khi chạy) hoặc ở Log; tìm theo cụm từ tiếng Anh trong cột đầu.

Thông báo	Nguyên nhân	Cách sửa
`Start the measurement first (HOME › Start)`	Bấm Run khi chưa đo	HOME › Start rồi Run
`no Measure network is connected`	Chưa có network nào được nối	Kiểm tra cấu hình kênh và interface, rồi HOME › Start
`is in the DBC of several networks … write "CAN1::…"`	Hai network cùng DBC, tên message thiếu network	Viết `"CAN1::Msg"`, hoặc chuột phải dòng lỗi » chọn cách sửa
`is not in the DBC … did you mean …?`	Gõ sai tên, hoặc DBC chưa nạp cho network	Sửa theo gợi ý; kiểm tra cửa sổ CAN Database
`several networks are connected … add channel="CAN1"`	`send` / `send_cyclic` theo CAN ID khi có nhiều network	Thêm `channel="CAN1"`, hoặc viết `"CAN1::0x123"`
`… waits, so it is only allowed inside @sequence`	Lệnh chờ trong handler hoặc trong hàm phụ	Chuyển vào `@sequence`; trong handler dùng timer
`acts on the bus: call it inside a handler such as @on_start`	`send`, `send_cyclic` viết ở cấp ngoài cùng của file	Đưa vào `@on_start`

Thông báo	Nguyên nhân	Cách sửa
`is a CAN FD message but network … is not in FD mode`	DBC khai báo message FD, network chưa bật FD	Bật FD cho network trong cấu hình kênh, hoặc dùng đúng DBC
`classic CAN carries at most 8 bytes, got …`	Gửi hơn 8 byte theo CAN ID trên network classic	Rút còn ⩽ 8 byte, hoặc bật FD cho network
`a CAN FD frame of … bytes … cannot go on network …`	`forward` frame FD > 8 byte sang network classic	Kiểm tra `msg.is_fd and msg.dlc > 8` trước khi forward (mục Gateway)
`is filled by the CheckSum generator on every frame`	`set_signal` vào signal CRC / Alive đang được tính tự động	Không ghi tay; muốn làm sai dùng `e2e_enable(..., crc=False)`
`e2e_message(…): … share bits` / `does not fit in … byte(s)`	CRC và Alive trùng bit; vị trí vượt quá độ dài message	Sửa vị trí; dùng hộp thoại E2E… để xem trên lưới bit
`the frame has … byte(s), the E2E needs …`	Frame gửi theo CAN ID ngắn hơn vị trí CRC / Alive	Gửi đủ số byte
`Second handler for …; both will run`	Hai handler cùng sự kiện	Gộp lại; hai signal cùng message thì dùng một `@on_message`
`Timer "…" has a handler but set_timer(…) is never called`	Có `@on_timer` mà không đặt timer	Thêm `set_timer(...)` ở `@on_start`
`…() took … ms: every other handler waited that long`	Handler chạy quá 20 ms	Tách việc dài ra timer, hoặc chuyển sang `@sequence`
`send on … failed: is the network connected?`	Interface mất kết nối hoặc network không hoạt động	Kiểm tra dây, interface, cấu hình kênh
`UDS needs the Diagnostic Setting` / ECU `not in Diagnostic Setting`	Mở Program không qua cửa sổ chính, hoặc tên ECU sai	Mở Program từ cửa sổ chính; viết đúng tên ECU như Diagnostic Setting
`…, which is not connected` (UDS)	Kênh Diagnostic của ECU chưa nối	Nối kênh đó trước khi Run

Phụ lục: ví dụ hoàn chỉnh

Năm chương trình dưới đây chép nguyên vào tab Code là chạy được, hoặc Open file riêng của từng cái: `A_c2_esc_simulation.py`, `B_c2_gateway_esc_ivc.py`, `C_c2_esc_uds_test.py`, `D_canfd_ecu_simulation.py`, `E_gateway_canfd_can.py`. A, B, C dùng DBC C2; D, E gửi theo CAN ID nên không cần DBC. Export CAPL: B, D, E xuất trọn vẹn; A và C xuất phần mô phỏng, còn unit Scenario và UDS test (có `@sequence`) chỉ chạy bằng Run.

Ví dụ	Bus	Network	Cho thấy
A. Mô phỏng ESC	CAN	1 network có DBC C2	E2E, chu kỳ DBC, phản ứng IVC, timer, giám sát timeout, phím, unit Scenario
B. Gateway ESC ↔ IVC	CAN	CAN1 (ESC), CAN2 (IVC), cùng DBC C2	`"*"` + handler riêng, sửa signal, chặn, CRC sai, thống kê
C. Test UDS ESC	CAN	1 network có DBC C2 + ECU `ESC` trong Diagnostic Setting	Rest-bus IVC, 8 bước PASS / FAIL, DTC, security access
D. Mô phỏng ECU CAN FD	CAN FD	`CANFD1` bật FD	Frame 32 / 64 byte, CRC-16 có Data ID, lệnh và phản hồi FD
E. Gateway CAN FD ↔ CAN	CAN FD + CAN	`CAN1` classic, `CAN2` FD	Quy tắc kiểu frame, lọc ID, cắt 8 byte

Giả định chung: `crc8()` của `crc_cal.cin` là CRC-8/SAE-J1850; tên network `CAN1`, `CAN2`, `CANFD1`; DID, NRC và mask DTC là ví dụ. Đổi theo thực tế trước khi dùng.

A. Mô phỏng ESC trên CAN (bus C2)

Chuẩn bị: DBC C2 nạp cho một network, ESC thật không nối vào bus. Chỉ cần mô phỏng thì tắt unit `Scenario`.

```
"""ESC simulation — Chassis 2 (DBC C2_CAN)

Chương trình đóng vai ESC trên bus C2 khi ESC thật không có trên bàn test.
  • Gửi 3 message của ESC theo chu kỳ của DBC: C2_ESC_ABS_Status 20 ms,
C2_ESC_Info 20 ms, C2_ESC_VAFS 40 ms
  • E2E như CAPL cũ: CRC ở byte 0 tính trên byte 1..7, Alive Counter ở byte 1
bit 0-3, đếm 0..14
```

```
  • Phản ứng với IVC: công tắc EPB, nút ESC OFF / Tow mode của MHU, đèn phanh
theo % bàn đạp phanh;
    C2_IVC_CVC_StsEpt im lặng > 200 ms -> ESC báo lỗi level 1
  • Unit "Scenario": @sequence chạy kịch bản tốc độ và các lỗi E2E / timeout
    (chỉ cần mô phỏng ESC: chuột phải tab "Scenario" > Turn off)

Phím (khi Run, cửa sổ Program có focus):
    c = CRC sai / đúng      a = Alive Counter đứng / chạy      f = lỗi ESC
level 1 bật / tắt

Export CAPL: unit Main (mô phỏng ESC) xuất được trọn vẹn khi DBC C2 đã nạp;
unit Scenario (@sequence, wait)
chỉ chạy bằng Run. Timeout của IVC dùng timer đặt lại ở mỗi frame (không dùng
msg_age) để CAPL cũng làm được.
Giả định: crc8() của crc_cal.cin là CRC-8/SAE-J1850. Nếu khác, chỉ cần sửa
e2e_config() bên dưới.
"""

# ═══════════════════════════════ E2E
# Đọc một lần khi chương trình nạp. Run, tab Panel và Export CAPL dùng chung
khai báo này.
e2e_config(crc="CRC-8/SAE-J1850", alive="0-14")
e2e_message("C2_ESC_ABS_Status", crc="CRC_C2_ESC_ABS_Status",
alive="ALIVE_C2_ESC_ABS_Status")
e2e_message("C2_ESC_Info", crc="CRC_C2_ESC_Info", alive="ALIVE_C2_ESC_Info")
e2e_message("C2_ESC_VAFS", crc="CRC_C2_ESC_VAFS", alive="ALIVE_C2_ESC_VAFS")

# ═══════════════════════════════ hằng số
EPB_RELEASED = 1                    # C2_EPB_Sts (VAL_ của DBC)
EPB_LOCKED = 2
EPB_RELEASING = 3
EPB_LOCKING = 4
EPB_MOVE_MS = 800                   # thời gian nhả / kẹp phanh tay
STANDSTILL_KMH = 0.5                # dưới mức này: xe đứng yên
IVC_TIMEOUT_MS = 200                # IVC im lặng lâu hơn: ESC báo lỗi level
1

# ═══════════════════════════════ biến (tab Var & Time / Panel)
speed = 0.0                         # km/h, đặt bằng set_speed()
epb = EPB_LOCKED                    # trạng thái phanh tay hiện tại
fault_level = 0                     # 0 No Fault, 1 Level 1, 2 Level 2 (phím
f, kịch bản)
ivc_lost = False                    # C2_IVC_CVC_StsEpt im lặng quá
IVC_TIMEOUT_MS
crc_bad = False
alive_stop = False
```

```
# ═══════════════════════════════ hàm phụ (không chờ)
def set_speed(kmh):
    """Tốc độ xe vào C2_ESC_ABS_Status: tốc độ + cờ đứng yên (frame chu kỳ kế
tiếp mang giá trị mới)."""
    speed = max(0.0, min(float(kmh), 300.0))
    set_signal("C2_ESC_ABS_Status", "C2_ESC_VehicleSpeed", speed)
    set_signal("C2_ESC_ABS_Status", "C2_ESC_VehicleStandstill", 1 if speed <
STANDSTILL_KMH else 0)


def set_epb(state):
    """Trạng thái phanh tay vào C2_ESC_VAFS."""
    epb = state
    set_signal("C2_ESC_VAFS", "C2_EPB_Sts", state)


def update_fault():
    """C2_ESC_Fault_Status = lỗi đặt bằng tay / kịch bản, ít nhất level 1 khi
mất IVC."""
    level = fault_level
    if ivc_lost and level < 1:
        level = 1
    set_signal("C2_ESC_Info", "C2_ESC_Fault_Status", level)


# ═══════════════════════════════ start / stop
@on_start
def start():
    set_speed(0)
    set_epb(EPB_LOCKED)
    set_signal("C2_ESC_ABS_Status", "C2_ESC_VehicleSpeedValid", "Valid")
    set_signal("C2_ESC_Info", "C2_ESC_Fault_Status", "No Fault")
    set_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC normal")
    send_cyclic("C2_ESC_ABS_Status")            # chu kỳ lấy từ DBC
(GenMsgCycleTime)
    send_cyclic("C2_ESC_Info")
    send_cyclic("C2_ESC_VAFS")
    set_timer("t_ivc_lost", 1000)               # IVC có 1 s để lên bus, sau
đó mỗi frame đặt lại 200 ms
    write("ESC simulation: running")


@on_stop
def stop():
```

```
    write("ESC simulation: stopped (speed %.1f km/h, EPB %d)", speed, epb)


# ═══════════════════════════════ IVC -> ESC
@on_signal("C2_IVC_CVC_HV_DrvSys_status", "C2_IVC_CVC_EPB_SwitchSta")
def epb_switch(value):
    """Nhấn công tắc EPB (1 = Pressed): đang kẹp -> nhả; đang nhả và xe đứng
yên -> kẹp."""
    if value != 1:
        return
    if epb == EPB_LOCKED:
        set_epb(EPB_RELEASING)
        set_timer("t_epb", EPB_MOVE_MS)
    elif epb == EPB_RELEASED and speed < STANDSTILL_KMH:
        set_epb(EPB_LOCKING)
        set_timer("t_epb", EPB_MOVE_MS)
    elif epb == EPB_RELEASED:
        set_signal("C2_ESC_VAFS", "C2_EPB_SysDspMsgReq", "Message #1")     #
ví dụ: "xe đang chạy, không kẹp"
        set_timer("t_epb_msg", 2000)


@on_timer("t_epb")
def epb_done():
    """Hết thời gian nhả / kẹp: chỉ hoàn tất thao tác đang dở (trạng thái đã
đổi nơi khác thì để nguyên)."""
    if epb == EPB_RELEASING:
        set_epb(EPB_RELEASED)
        write("EPB: Released")
    elif epb == EPB_LOCKING:
        set_epb(EPB_LOCKED)
        write("EPB: Locked")


@on_timer("t_epb_msg")
def epb_msg_off():
    set_signal("C2_ESC_VAFS", "C2_EPB_SysDspMsgReq", "Off")


@on_message("C2_IVC_CVC_StsEpt")
def ivc_sts(msg):
    """Mỗi frame IVC nhận được (20 ms): đèn phanh khi bàn đạp phanh > 5 %, và
đặt lại timer timeout 200 ms.
    Một @on_message cho cả hai việc (CAPL chỉ có một handler cho mỗi
message)."""
    if msg.signal("C2_IVC_CVC_BrkPedalPct") > 5:
```

```
        set_signal("C2_ESC_VAFS", "C2_ESC_BrkLampReq", 1)
    else:
        set_signal("C2_ESC_VAFS", "C2_ESC_BrkLampReq", 0)
    set_timer("t_ivc_lost", IVC_TIMEOUT_MS)
    if ivc_lost:
        ivc_lost = False
        update_fault()
        write("IVC: C2_IVC_CVC_StsEpt có lại")


@on_timer("t_ivc_lost")
def ivc_timeout():
    """C2_IVC_CVC_StsEpt im lặng > 200 ms: ESC báo lỗi level 1, như ESC
thật."""
    ivc_lost = True
    update_fault()
    write("IVC timeout: C2_IVC_CVC_StsEpt mất > %d ms", IVC_TIMEOUT_MS)


@on_message("C2_MHU_CarMode_Switch")
def car_mode(msg):
    """Nút ESC OFF (2 = OFF, 1 = ON, 0 = không bấm) và Tow mode (chỉ vào khi
xe đứng yên).
    Hai signal cùng một message: một @on_message đọc cả hai."""
    sw = msg.signal("C2_MHU_ESCOFF_Sw")
    if sw == 2:
        set_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC-OFF")
    elif sw == 1:
        set_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC normal")
    if msg.signal("C2_MHU_TowMode_Req") == 1 and speed < STANDSTILL_KMH:
        set_signal("C2_ESC_VAFS", "C2_ESC_TowMode_Feedback", "Active")
    else:
        set_signal("C2_ESC_VAFS", "C2_ESC_TowMode_Feedback", "Inactive")


# ═══════════════════════════════ phím: tiêm lỗi bằng tay
@on_key("c")
def key_crc():
    """CRC sai / đúng trên C2_ESC_ABS_Status (crc=False: byte 0 không được
tính)."""
    crc_bad = not crc_bad
    if crc_bad:
        e2e_enable("C2_ESC_ABS_Status", crc=False)
        write("C2_ESC_ABS_Status: CRC sai")
    else:
        e2e_enable("C2_ESC_ABS_Status", crc=True)
```

```
        write("C2_ESC_ABS_Status: CRC đúng")


@on_key("a")
def key_alive():
    """Alive Counter đứng / chạy trên C2_ESC_ABS_Status (CRC vẫn đúng)."""
    alive_stop = not alive_stop
    if alive_stop:
        e2e_enable("C2_ESC_ABS_Status", alive=False)
        write("C2_ESC_ABS_Status: Alive Counter đứng")
    else:
        e2e_enable("C2_ESC_ABS_Status", alive=True)
        write("C2_ESC_ABS_Status: Alive Counter chạy")


@on_key("f")
def key_fault():
    if fault_level:
        fault_level = 0
    else:
        fault_level = 1
    update_fault()
    write("ESC fault level: %d", fault_level)


# ==== unit: Scenario ====
# Kịch bản cho IVC. @sequence chạy sau @on_start, trong luồng riêng: wait() /
expect_signal() chỉ dùng ở đây.

@sequence("ESC scenario")
def scenario():
    write("[1] Đứng yên, EPB kẹp: 2 s")
    wait(2000)

    write("[2] Nhả EPB, tăng tốc 0 -> 60 km/h trong 6 s")
    set_epb(EPB_RELEASED)
    for kmh in range(0, 61):
        set_speed(kmh)
        wait(100)

    write("[3] Giữ 60 km/h 3 s. IVC báo: HV %d, số %d",
          get_signal("C2_IVC_CVC_HV_DrvSys_status",
"C2_IVC_CVC_HVsystem_status"),
          get_signal("C2_IVC_CVC_HV_DrvSys_status", "C2_IVC_CVC_ACTGear"))
    wait(3000)
```

```
write("[4] Phanh 60 -> 0 km/h trong 3 s")
for kmh in range(60, -1, -2):
    set_speed(kmh)
    wait(100)
wait(1000)

write("[5] CRC sai 1 s trên C2_ESC_ABS_Status")
e2e_enable("C2_ESC_ABS_Status", crc=False)
wait(1000)
e2e_enable("C2_ESC_ABS_Status", crc=True)
wait(1000)

write("[6] Alive Counter đứng 1 s")
e2e_enable("C2_ESC_ABS_Status", alive=False)
wait(1000)
e2e_enable("C2_ESC_ABS_Status", alive=True)
wait(1000)

write("[7] Mất C2_ESC_ABS_Status 500 ms (timeout)")
stop_cyclic("C2_ESC_ABS_Status")
wait(500)
send_cyclic("C2_ESC_ABS_Status")
wait(1000)

write("[8] Tốc độ không hợp lệ 1 s")
set_signal("C2_ESC_ABS_Status", "C2_ESC_VehicleSpeedValid", "Invalid")
wait(1000)
set_signal("C2_ESC_ABS_Status", "C2_ESC_VehicleSpeedValid", "Valid")

write("[9] ESC lỗi level 2 trong 2 s")
fault_level = 2
update_fault()
wait(2000)
fault_level = 0
update_fault()

write("[10] Kẹp lại EPB. Kịch bản xong")
set_epb(EPB_LOCKED)
```

B. Gateway ESC ↔ IVC trên CAN

Chuẩn bị: bus C2 cắt đôi, ESC nối kênh của `CAN1`, phần còn lại (IVC) nối kênh của `CAN2`; DBC C2 nạp cho cả hai network. Network tên khác thì Find / Replace `CAN1`, `CAN2`.

```
"""Gateway ESC <-> IVC — Chassis 2 (DBC C2_CAN)

Tool đứng giữa ESC và IVC: bus C2 cắt làm hai, cả hai network nạp DBC C2
trong CAN Database.
    CAN1 = phía ESC          CAN2 = phía IVC
    (network tên khác: Find / Replace "CAN1" và "CAN2" trong cả file)
  • Mọi frame đi qua hai chiều nguyên vẹn: ID, DLC, dữ liệu, Alive và CRC do
ECU gửi
  • Can thiệp bằng phím:
        1 = tốc độ giả 120 km/h, ESC -> IVC       2 = chặn ESC OFF, IVC ->
ESC
        3 = bỏ C2_ESC_Info (IVC thấy timeout)     4 = CRC sai trên
C2_ESC_VAFS
        0 = tắt hết, đi thẳng
    Khi sửa signal: CRC được tính lại (e2e_message), Alive Counter giữ nguyên
như ECU gửi.

Handler riêng của một message thắng handler "*" của network đó (giống CAPL
"on message CAN1.*").
Frame do tool gửi ra (Tx) không vào lại handler, nên không có vòng lặp.
"""

# ═══════════════════════════════ E2E
# Chỉ cần cho message mà gateway sửa signal: CRC tính lại theo khai báo của
network đích.
e2e_config(crc="CRC-8/SAE-J1850", alive="0-14")
e2e_message("CAN2::C2_ESC_ABS_Status", crc="CRC_C2_ESC_ABS_Status",
alive="ALIVE_C2_ESC_ABS_Status")
e2e_message("CAN1::C2_MHU_CarMode_Switch", crc="CRC_C2_MHU_CarMode_Switch",
alive="ALIVE_C2_MHU_CarMode_Switch")

# ═══════════════════════════════ hằng số
FAKE_SPEED_KMH = 120
STAT_MS = 5000                      # chu kỳ ghi thống kê vào Log

# ═══════════════════════════════ biến
fake_speed = False
block_esc_off = False
drop_esc_info = False
bad_crc_vafs = False
n_esc = 0                           # frame ESC -> IVC
n_ivc = 0                           # frame IVC -> ESC
n_changed = 0                       # frame gateway đã sửa hoặc bỏ


# ═══════════════════════════════ start
```

```
@on_start
def start():
    set_timer("t_stat", STAT_MS)
    write("Gateway: ESC (CAN1) <-> IVC (CAN2)")


# ═══════════════════════════════ đi thẳng
@on_message("*", channel="CAN1")
def esc_to_ivc(msg):
    """Mọi frame của ESC không có handler riêng: sang IVC nguyên vẹn."""
    forward(msg, "CAN2")
    n_esc += 1


@on_message("*", channel="CAN2")
def ivc_to_esc(msg):
    """Mọi frame của IVC không có handler riêng: sang ESC nguyên vẹn."""
    forward(msg, "CAN1")
    n_ivc += 1


# ═══════════════════════════════ ESC -> IVC: can thiệp
@on_message("CAN1::C2_ESC_ABS_Status")
def abs_status(msg):
    """Tốc độ giả: đổi 2 signal; CRC tính lại, Alive giữ của ESC."""
    if fake_speed:
        forward(msg, "CAN2", {"C2_ESC_VehicleSpeed": FAKE_SPEED_KMH,
"C2_ESC_VehicleStandstill": 0})
        n_changed += 1
    else:
        forward(msg, "CAN2")
    n_esc += 1


@on_message("CAN1::C2_ESC_Info")
def esc_info(msg):
    """Bỏ message: không forward, IVC thấy C2_ESC_Info mất."""
    if drop_esc_info:
        n_changed += 1
        return
    forward(msg, "CAN2")
    n_esc += 1


@on_message("CAN1::C2_ESC_VAFS")
def esc_vafs(msg):
```

```
    """CRC sai: ghi chính signal CRC (byte 0) đảo bit. Signal CRC năm trong
danh sách sửa thì không bị tính lại."""
    if bad_crc_vafs:
        forward(msg, "CAN2", {"CRC_C2_ESC_VAFS": msg.byte(0) ^ 0xFF})
        n_changed += 1
    else:
        forward(msg, "CAN2")
    n_esc += 1


# ═══════════════════════════════ IVC -> ESC: can thiệp
@on_message("CAN2::C2_MHU_CarMode_Switch")
def car_mode(msg):
    """Chặn ESC OFF: 2 (OFF) đổi thành "No Action" trước khi tới ESC."""
    if block_esc_off and msg.signal("C2_MHU_ESCOFF_Sw") == 2:
        forward(msg, "CAN1", {"C2_MHU_ESCOFF_Sw": "No Action"})
        n_changed += 1
    else:
        forward(msg, "CAN1")
    n_ivc += 1


# ═══════════════════════════════ theo dõi
@on_signal("CAN1::C2_ESC_VAFS", "C2_EPB_Sts")
def epb_changed(value):
    write("ESC: C2_EPB_Sts -> %d", value)


@on_signal("CAN1::C2_ESC_Info", "C2_STAT_DSC")
def dsc_changed(value):
    write("ESC: C2_STAT_DSC -> %d", value)


@on_timer("t_stat")
def stat():
    write("ESC->IVC %d, IVC->ESC %d, đã sửa/bỏ %d | tốc độ ESC %.1f km/h",
          n_esc, n_ivc, n_changed, get_signal("CAN1::C2_ESC_ABS_Status",
"C2_ESC_VehicleSpeed"))
    set_timer("t_stat", STAT_MS)


# ═══════════════════════════════ phím
@on_key("1")
def key_speed():
    fake_speed = not fake_speed
    if fake_speed:
```

```
        write("Tốc độ giả %d km/h -> IVC: ON", FAKE_SPEED_KMH)
    else:
        write("Tốc độ giả -> IVC: OFF")


@on_key("2")
def key_esc_off():
    block_esc_off = not block_esc_off
    if block_esc_off:
        write("Chặn ESC OFF -> ESC: ON")
    else:
        write("Chặn ESC OFF -> ESC: OFF")


@on_key("3")
def key_drop():
    drop_esc_info = not drop_esc_info
    if drop_esc_info:
        write("Bỏ C2_ESC_Info: ON")
    else:
        write("Bỏ C2_ESC_Info: OFF")


@on_key("4")
def key_crc():
    bad_crc_vafs = not bad_crc_vafs
    if bad_crc_vafs:
        write("CRC sai C2_ESC_VAFS: ON")
    else:
        write("CRC sai C2_ESC_VAFS: OFF")


@on_key("0")
def key_reset():
    fake_speed = False
    block_esc_off = False
    drop_esc_info = False
    bad_crc_vafs = False
    write("Gateway: đi thẳng, không can thiệp")
```

C. Test UDS với ESC

Chuẩn bị: ESC thật nối bus C2, DBC C2 nạp cho network đó, ECU `ESC` có trong Diagnostic Setting (ID request / response, kênh Diagnostic, DLL Security Access). Unit `Main` giả lập IVC, unit `UDS test` chạy các bước.

```
"""ESC UDS test — ESC thật trên bus C2 (DBC C2_CAN)

ESC cần thấy IVC trên bus, nếu không sẽ ghi DTC mất giao tiếp. Vì vậy chương
trình gồm hai unit:
  • Main: giả lập IVC. Gửi 8 message IVC_C2 của DBC với E2E như CAPL cũ: điện
ON, HV ReadyToDrive, số P,
    bàn đạp hợp lệ, cửa đóng, dây an toàn đã cài
  • UDS test: @sequence "ESC UDS test", mỗi bước một dòng PASS / FAIL trong
Log, cuối cùng in tổng kết
       1. ESC có mặt trên bus              2. default session, đọc DID F190 /
F187 / F195
       3. NRC mong đợi                     4. extended session, Tester
Present, Security Access
       5. xoá DTC rồi đọc lại              6. DTC mất giao tiếp: dừng
C2_IVC_CVC_StsEpt 2 s
       7. chức năng: nút ESC OFF của MHU   8. ECU reset, ESC về default
session

Cần: ECU "ESC" trong Diagnostic Setting (ID request / response, kênh
Diagnostic, DLL Security Access).
DID, NRC và mask DTC ở đây là ví dụ: đổi theo Diagnostic Spec của ESC.
Quy tắc: wait / wait_for_message / expect_signal / uds_request /
security_access chỉ gọi TRỰC TIẾP trong
@sequence. Hàm phụ chỉ đánh giá kết quả, không chờ (Issue tab báo lỗi nếu hàm
phụ chờ).
Export CAPL: unit Main (IVC giả) xuất được khi DBC C2 đã nạp, dùng làm rest-
bus trong CANoe; unit UDS test chỉ
chạy bằng Run.
"""

# ═══════════════════════════════ E2E của IVC giả
e2e_config(crc="CRC-8/SAE-J1850", alive="0-14")
e2e_message("C2_IVC_CVC_CLAMP_STAT", crc="CRC_C2_IVC_CVC_CLAMP_STAT",
alive="ALIVE_C2_IVC_CVC_CLAMP_STAT")
e2e_message("C2_IVC_CVC_HV_DrvSys_status",
crc="CRC_C2_IVC_CVC_HV_DrvSys_status",
            alive="ALIVE_C2_IVC_CVC_HV_DrvSys_statu")
e2e_message("C2_IVC_CVC_StsEpt", crc="CRC_C2_IVC_CVC_StsEpt",
alive="ALIVE_C2_IVC_CVC_StsEpt")
e2e_message("C2_IVC_CVC_ESC02", crc="CRC_C2_IVC_CVC_ESC02",
alive="ALIVE_C2_IVC_CVC_ESC02")
e2e_message("C2_MHU_CarMode_Switch", crc="CRC_C2_MHU_CarMode_Switch",
alive="ALIVE_C2_MHU_CarMode_Switch")
e2e_message("C2_SS_DriveReq", crc="CRC_C2_SS_DriveReq",
alive="ALIVE_C2_SS_DriveReq")
e2e_message("C2_BCM_DoorStatus", crc="CRC_C2_BCM_DoorStatus",
```

```
alive="ALIVE_C2_BCM_DoorStatus")
e2e_message("C2_ACU_OCCUPANT_STATUS", crc="CRC_C2_ACU_OCCUPANT_STATUS",
alive="ALIVE_C2_ACU_OCCUPANT_STATUS")

# ═══════════════════════════════ IVC giả
@on_start
def start():
    """Trạng thái xe cho chẩn đoán: điện ON, HV ReadyToDrive, số P, đứng
yên."""
    set_signal("C2_IVC_CVC_CLAMP_STAT", "C2_Vehicle_Operation_State", "ON
(Driver Present)")
    set_signal("C2_IVC_CVC_HV_DrvSys_status", "C2_IVC_CVC_HVsystem_status",
"ReadyToDrive")
    set_signal("C2_IVC_CVC_HV_DrvSys_status", "C2_IVC_CVC_ACTGear", "P")
    set_signal("C2_IVC_CVC_HV_DrvSys_status", "C2_IVC_CVC_ACTGearValid",
"Valid")
    set_signal("C2_IVC_CVC_StsEpt", "C2_IVC_CVC_AccPedal_Valid", "Valid")
    set_signal("C2_IVC_CVC_StsEpt", "C2_IVC_CVC_BrkPedalValid", "Valid")
    set_signal("C2_IVC_CVC_ESC02", "C2_IVC_CVC_F_TrqCurrWhlValid", "valid")
    set_signal("C2_ACU_OCCUPANT_STATUS", "C2_ACU_Seat_Belt_Status_Driver",
"Buckled")
    send_cyclic("C2_IVC_CVC_CLAMP_STAT")        # chu kỳ lấy từ DBC
(GenMsgCycleTime)
    send_cyclic("C2_IVC_CVC_HV_DrvSys_status")
    send_cyclic("C2_IVC_CVC_StsEpt")
    send_cyclic("C2_IVC_CVC_ESC02")
    send_cyclic("C2_MHU_CarMode_Switch")
    send_cyclic("C2_SS_DriveReq")
    send_cyclic("C2_BCM_DoorStatus")
    send_cyclic("C2_ACU_OCCUPANT_STATUS")
    write("IVC simulation: 8 message")


# ==== unit: UDS test ====
ECU = "ESC"                                     # tên ECU trong Diagnostic
Setting
DTC_MASK = "09"                                 # 19 02: testFailed |
confirmedDTC
n_pass = 0
n_fail = 0


# ═══════════════════════════════ hàm phụ (không chờ)
def step(name, ok, detail=""):
    """Một bước của test: PASS / FAIL vào Log, trả lại ok."""
    if ok:
```

```
        n_pass += 1
    else:
        n_fail += 1
    write("%s  %s  %s", "PASS" if ok else "FAIL", name, detail)
    return ok


def check_positive(name, r):
    """Phản hồi dương?"""
    return step(name, r.positive, r.hex() or r.error)


def check_nrc(name, r, code):
    """Phản hồi âm với đúng mã NRC?"""
    return step(name, not r.positive and r.nrc == code, r.hex() or r.error)


def dtc_text(b1, b2, b3):
    """3 byte DTC -> dạng SAE, ví dụ 5A 12 34 -> "C1A12-34"."""
    return "%s%X%X%02X-%02X" % ("PCBU"[b1 >> 6], (b1 >> 4) & 0x3, b1 & 0xF,
b2, b3)


def dtc_list(r):
    """Phản hồi 59 02 <mask> | DTC H M L, status | ... -> danh sách DTC; None
nếu phản hồi âm."""
    if not r.positive:
        return None
    out = []
    i = 3
    while i + 3 < len(r.data):
        out.append(dtc_text(r.byte(i), r.byte(i + 1), r.byte(i + 2)))
        i += 4
    return out


# ═══════════════════════════════ test
@sequence("ESC UDS test")
def uds_test():
    write("===== ESC UDS test =====")
    wait(1000)                                  # ESC khởi động và thấy IVC

    # 1. ESC có mặt
    step("ESC gửi C2_ESC_Info", wait_for_message("C2_ESC_Info", timeout=500)
is not None)
```

```
    # 2. default session, đọc DID
    check_positive("10 01 default session", uds_request(ECU, "10 01"))
    for did, label in (("F1 90", "VIN"), ("F1 87", "part number"), ("F1 95",
"SW version")):
        r = uds_request(ECU, "22 " + did)
        step("22 %s %s" % (did, label), r.positive, r.ascii(3) if r.positive
else (r.hex() or r.error))

    # 3. NRC mong đợi
    check_nrc("22 FF FF -> NRC 31 requestOutOfRange", uds_request(ECU, "22 FF
FF"), 0x31)
    check_nrc("27 01 ở default session -> NRC 7F", uds_request(ECU, "27 01"),
0x7F)

    # 4. extended session, Tester Present, Security Access
    check_positive("10 03 extended session", uds_request(ECU, "10 03"))
    tester_present(ECU, True)                   # 3E 80 mỗi 2 s khi không có
request
    step("27 security access level 1", bool(security_access(ECU, 1)))

    # 5. xoá DTC rồi đọc lại
    check_positive("14 FF FF FF clear DTC", uds_request(ECU, "14 FF FF FF"))
    wait(500)
    dtcs = dtc_list(uds_request(ECU, "19 02 " + DTC_MASK))
    step("19 02 %s sau khi xoá: không có DTC" % DTC_MASK, dtcs == [], ",
".join(dtcs or []))

    # 6. DTC mất giao tiếp với IVC
    write("Dừng C2_IVC_CVC_StsEpt 2 s ...")
    stop_cyclic("C2_IVC_CVC_StsEpt")
    wait(2000)
    lost = dtc_list(uds_request(ECU, "19 02 " + DTC_MASK))
    step("19 02 %s khi mất C2_IVC_CVC_StsEpt: có DTC" % DTC_MASK, bool(lost),
", ".join(lost or []))
    send_cyclic("C2_IVC_CVC_StsEpt")
    wait(1000)
    check_positive("14 FF FF FF clear DTC", uds_request(ECU, "14 FF FF FF"))

    # 7. chức năng: nút ESC OFF của MHU
    set_signal("C2_MHU_CarMode_Switch", "C2_MHU_ESCOFF_Sw", "OFF")
    step("ESC OFF -> C2_STAT_DSC = ESC-OFF",
         expect_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC-OFF",
timeout=1000))
    set_signal("C2_MHU_CarMode_Switch", "C2_MHU_ESCOFF_Sw", "ON")
    step("ESC ON -> C2_STAT_DSC = ESC normal",
         expect_signal("C2_ESC_Info", "C2_STAT_DSC", "ESC normal",
```

```
timeout=1000))
    set_signal("C2_MHU_CarMode_Switch", "C2_MHU_ESCOFF_Sw", "No Action")

    # 8. ECU reset
    check_positive("11 01 hard reset", uds_request(ECU, "11 01"))
    tester_present(ECU, False)
    wait(1500)
    step("ESC gửi lại sau reset", wait_for_message("C2_ESC_Info",
timeout=1000) is not None)
    r = uds_request(ECU, "22 F1 86")
    step("22 F1 86 sau reset = default session (01)", r.positive and
r.byte(3) == 0x01, r.hex() or r.error)

    write("===== %d PASS, %d FAIL =====", n_pass, n_fail)
```

D. Mô phỏng ECU trên CAN FD

Chuẩn bị: network `CANFD1` bật chế độ FD trong cấu hình kênh; không cần DBC. Để thử lệnh, gửi `0x2A0` với byte 0 = `01` từ Send CAN hoặc từ node khác; tab Bus hiện `0x1A0` (32 byte, 10 ms), `0x1B0` (64 byte, 100 ms) và phản hồi `0x2A8` (12 byte), cả ba đều là frame FD có BRS. Cả 6 handler xuất được sang CAPL (tick **FD** cho `CANFD1` trong Export CAPL), vì chương trình viết byte thẳng vào list toàn cục, mỗi `write` một câu cố định và `e2e_enable` nhận `True` / `False` viết sẵn.

```
"""CAN FD ECU simulation — frame gửi theo CAN ID trên network FD (không cần
DBC)

Network CANFD1 phải bật chế độ FD (data bitrate) trong cấu hình kênh.
  • 0x1A0, 32 byte, 10 ms : byte 4-5 tốc độ (0.01 km/h, byte thấp trước),
byte 6 trạng thái (1 = chạy)
                            E2E: CRC-16/CCITT-FALSE ở byte 0-1 (byte 0 = byte
cao), Data ID 0x1A0,
                                 Alive Counter ở byte 2 bit 0-3, đếm 0..14
  • 0x1B0, 64 byte, 100 ms: byte 4-7 bộ đếm (dword, byte thấp trước)
                            E2E: CRC-8/SAE-J1850 ở byte 0, Alive Counter ở
byte 1 bit 0-3, đếm 0..14
  • Nhận lệnh 0x2A0 (byte 0: 1 = chạy, 0 = dừng), trả lời ngay bằng 0x2A8 (12
byte)
  • Mỗi 100 ms: chạy thì tăng tốc 1 km/h tới 120 km/h, dừng thì giảm 2 km/h
về 0
Phím: c = CRC 0x1A0 sai / đúng      s = dừng / gửi lại 0x1A0 (thử timeout ở
ECU nhận)

Mọi frame theo CAN ID trên network FD là CAN FD có BRS, đệm lên 8/12/…/64
byte (0x2A8: 12 byte).
```

```
Xuất được trọn vẹn sang CAPL (Export CAPL…): cột FD của CANFD1 phải được tick
(tool tự tick theo cấu hình kênh).
Viết theo kiểu CAPL hiểu được: byte ghi thẳng vào list toàn cục, mỗi write()
một câu cố định, e2e_enable()
nhận True / False viết sẵn.
CAN ID, độ dài, vị trí byte và thuật toán chỉ là ví dụ: đổi theo spec của
ECU.
"""

# ═══════════════════════════════ E2E (frame theo CAN ID: cần network=)
e2e_message(0x1A0, network="CANFD1", crc="bytes0-1 big", alive="byte2 bit0-
3",
            algo="CRC-16/CCITT-FALSE", data_id=0x1A0, alive_range=(0, 14))
e2e_message(0x1B0, network="CANFD1", crc="byte0", alive="byte1 bit0-3",
            algo="CRC-8/SAE-J1850", alive_range=(0, 14))

# ═══════════════════════════════ hằng số
SPEED_MAX_KMH = 120
TICK_MS = 100
PERIOD_1A0_MS = 10
PERIOD_1B0_MS = 100

# ═══════════════════════════════ biến
running = False
speed: word = 0                     # km/h
counter: dword = 0
crc_bad = False
stopped_1a0 = False
buf_1a0 = [0] * 32                  # dữ liệu của 0x1A0 (byte 0-2: E2E tự
điền khi gửi)
buf_1b0 = [0] * 64                  # dữ liệu của 0x1B0 (byte 0-1: E2E tự
điền khi gửi)


# ═══════════════════════════════ hàm phụ (không chờ)
def update_1a0():
    """Tốc độ (0.01 km/h, byte thấp trước) và trạng thái vào 0x1A0; gửi lại
send_cyclic để frame kế tiếp mang
    dữ liệu mới (cùng chu kỳ: chu kỳ đang chạy giữ nguyên)."""
    raw = speed * 100
    buf_1a0[4] = raw & 0xFF
    buf_1a0[5] = (raw >> 8) & 0xFF
    if running:
        buf_1a0[6] = 1
    else:
        buf_1a0[6] = 0
```

```
    if not stopped_1a0:
        send_cyclic(0x1A0, PERIOD_1A0_MS, buf_1a0, channel="CANFD1")


def update_1b0():
    """Bộ đếm (dword, byte thấp trước) vào 0x1B0."""
    buf_1b0[4] = counter & 0xFF
    buf_1b0[5] = (counter >> 8) & 0xFF
    buf_1b0[6] = (counter >> 16) & 0xFF
    buf_1b0[7] = (counter >> 24) & 0xFF
    send_cyclic(0x1B0, PERIOD_1B0_MS, buf_1b0, channel="CANFD1")


# ================================ start / stop
@on_start
def start():
    update_1a0()
    update_1b0()
    set_timer("t_tick", TICK_MS)
    write("CAN FD simulation: 0x1A0 (32 byte, 10 ms), 0x1B0 (64 byte, 100 ms)
trên CANFD1")


@on_stop
def stop():
    write("CAN FD simulation: dừng ở %d km/h, bộ đếm %d", speed, counter)


# ================================ lệnh và phản hồi
@on_message("CANFD1::0x2A0")
def command(msg):
    """Lệnh: byte 0 = 1 chạy, khác 1 dừng. Trả lời 0x2A8: 2A, lệnh đã nhận,
10 byte 00 (12 byte)."""
    running = msg.byte(0) == 1
    send(0x2A8, [0x2A, msg.byte(0), 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
channel="CANFD1")
    if running:
        write("Lệnh 0x2A0: chạy (FD %d, %d byte)", msg.is_fd, msg.dlc)
    else:
        write("Lệnh 0x2A0: dừng (FD %d, %d byte)", msg.is_fd, msg.dlc)
    update_1a0()


# ================================ chu kỳ 100 ms
@on_timer("t_tick")
def tick():
```

```
    if running:
        if speed < SPEED_MAX_KMH:
            speed += 1
    elif speed >= 2:
        speed -= 2
    else:
        speed = 0
    counter += 1
    update_1a0()
    update_1b0()
    set_timer("t_tick", TICK_MS)


# ═══════════════════════════════ phím: tiêm lỗi
@on_key("c")
def key_crc():
    """CRC-16 của 0x1A0 sai / đúng (crc=False: byte 0-1 không được tính)."""
    crc_bad = not crc_bad
    if crc_bad:
        e2e_enable(0x1A0, crc=False, channel="CANFD1")
        write("0x1A0: CRC sai")
    else:
        e2e_enable(0x1A0, crc=True, channel="CANFD1")
        write("0x1A0: CRC đúng")


@on_key("s")
def key_stop():
    """Dừng 0x1A0 để thử timeout ở ECU nhận; bấm lần nữa để gửi lại."""
    stopped_1a0 = not stopped_1a0
    if stopped_1a0:
        stop_cyclic(0x1A0, channel="CANFD1")
        write("0x1A0: dừng gửi")
    else:
        update_1a0()
        write("0x1A0: gửi lại")
```

E. Gateway CAN FD ↔ CAN classic

Chuẩn bị: `CAN1` là bus classic, `CAN2` là bus FD (bật FD trong cấu hình kênh); không cần DBC. Cả 8 handler xuất được sang CAPL. Trong Export CAPL, cột **FD** của `CAN2` phải được tick (tool tự tick theo cấu hình kênh). Khi đó, trong `on message CAN2.*`, node tự đặt `fwd.FDF = 0;` và `fwd.BRS = 0;` trước `output(fwd)` để frame ra CAN1 là classic. Nếu frame FD dài hơn 8 byte, node in ra Write thay vì gửi, giống Run.

```
"""Gateway CAN FD (CAN2) <-> CAN classic (CAN1)

CAN1 = bus CAN classic, CAN2 = bus CAN FD (bật chế độ FD trong cấu hình
kênh). Không cần DBC.
  • CAN1 -> CAN2: mọi frame đi nguyên vẹn; trên CAN2 chúng vẫn là frame
classic
  • CAN2 -> CAN1: frame classic và frame FD dài ≤ 8 byte đi qua (frame FD
thành classic);
                  frame FD dài hơn 8 byte không vừa CAN classic:
                    - ID được cắt (0x1B0): giữ 8 byte đầu rồi gửi (chỉ dùng
cho frame KHÔNG có E2E,
                      vì CRC tính trên cả frame dài sẽ sai sau khi cắt)
                    - ID khác: bỏ và đếm
  • ID bị chặn (0x7DF) không đi qua chiều nào
  • Log thống kê mỗi 5 s
Phím: b = bật / tắt chặn      t = bật / tắt cắt 8 byte      0 = xoá bộ đếm
ID chặn (0x7DF) và ID cắt (0x1B0) chỉ là ví dụ: danh sách ID viết thẳng trong
điều kiện, ví dụ
msg.id in (0x7DF, 0x7E0), để Export CAPL dịch được.

Export CAPL: cả 8 handler xuất được, để chạy trong CANoe khi cần độ trễ thấp.
Cột FD của CAN2 trong hộp thoại
Export CAPL phải được tick (tool tự tick theo cấu hình kênh). Khi đó node tự
làm như Run: frame FD ≤ 8 byte
forward ra CAN1 thành classic (FDF = 0, BRS = 0), frame FD dài hơn không được
gửi (in ra Write).
"""

# ═══════════════════════════════ hằng số
STAT_MS = 5000

# ═══════════════════════════════ biến
use_block = True
use_truncate = True
n_to_fd = 0                          # CAN1 -> CAN2
n_to_classic = 0                     # CAN2 -> CAN1
n_truncated = 0
n_dropped = 0
n_blocked = 0


# ═══════════════════════════════ start / stop
@on_start
def start():
    set_timer("t_stat", STAT_MS)
    write("Gateway: CAN1 (classic) <-> CAN2 (FD)")
```

```
@on_stop
def stop():
    write("Gateway dừng: CAN1->CAN2 %d | CAN2->CAN1 %d, cắt %d, bỏ %d | chặn
%d",
          n_to_fd, n_to_classic, n_truncated, n_dropped, n_blocked)


# ================================ CAN1 (classic) -> CAN2 (FD)
@on_message("*", channel="CAN1")
def classic_to_fd(msg):
    if use_block and msg.id in (0x7DF,):        # ID bị chặn
        n_blocked += 1
        return
    forward(msg, "CAN2")                        # vẫn là frame classic trên
CAN2
    n_to_fd += 1


# ================================ CAN2 (FD) -> CAN1 (classic)
@on_message("*", channel="CAN2")
def fd_to_classic(msg):
    if use_block and msg.id in (0x7DF,):        # ID bị chặn
        n_blocked += 1
        return
    if msg.is_fd and msg.dlc > 8:
        if use_truncate and msg.id in (0x1B0,):     # ID được cắt: frame
không có E2E
            forward(msg, "CAN1", data=[msg.byte(0), msg.byte(1), msg.byte(2),
msg.byte(3),
                                       msg.byte(4), msg.byte(5), msg.byte(6),
msg.byte(7)])   # 8 byte đầu, classic
            n_truncated += 1
        else:
            n_dropped += 1
        return
    forward(msg, "CAN1")                        # classic, hoặc FD ≤ 8 byte -
> classic
    n_to_classic += 1


# ================================ thống kê
@on_timer("t_stat")
def stat():
    write("CAN1->CAN2 %d | CAN2->CAN1 %d, cắt %d, bỏ %d | chặn %d",
```

```
          n_to_fd, n_to_classic, n_truncated, n_dropped, n_blocked)
    set_timer("t_stat", STAT_MS)


# ═══════════════════════════════ phím
@on_key("b")
def key_block():
    use_block = not use_block
    if use_block:
        write("Chặn 0x7DF: ON")
    else:
        write("Chặn 0x7DF: OFF")


@on_key("t")
def key_truncate():
    use_truncate = not use_truncate
    if use_truncate:
        write("Cắt 8 byte cho 0x1B0: ON")
    else:
        write("Cắt 8 byte cho 0x1B0: OFF")


@on_key("0")
def key_reset():
    n_to_fd = 0
    n_to_classic = 0
    n_truncated = 0
    n_dropped = 0
    n_blocked = 0
    write("Đã xoá bộ đếm")
```